Foundations of Language Modeling

Chapter 1: Introduction to Language Models

1 Introduction to Language Modeling

The fundamental objective of a language model is to model the probability distribution of a sequence of discrete symbols (tokens) and, by extension, to predict the next token given a sequence of preceding tokens.

Definition 1.1 (Language Model)**.** *Let $\mathcal{V}$ be a finite set of symbols, called a* ***vocabulary****. Let $X = (x_1, x_2, \ldots, x_T)$ be a sequence of tokens such that $x_t \in \mathcal{V}$ for all t. An autoregressive language model defines a joint probability distribution $P(X)$ over the sequence by factoring it using the chain rule of probability:*

$$P(x_1, x_2, \ldots, x_T) = \prod_{t=1}^{T} P(x_t \mid x_1, \ldots, x_{t-1}) \tag{1}$$

where $P(x_1 \mid x_0)$ is defined simply as the marginal prior $P(x_1)$.

Intuition: To generate text, the model acts as an algorithmic continuation mechanism. Given a prompt $(x_1, \ldots, x_t)$, the model computes the conditional probability distribution for x_{t+1} over all tokens in $\mathcal{V}$. We then sample from this distribution to select x_{t+1}, append it to our sequence, and repeat the process ad infinitum.

2 Tokenization and Vocabularies

Before a neural network can process text, continuous strings must be converted into discrete integer arrays. This process is called **tokenization**.

Definition 2.1 (Tokenization Mapping)**.** *We define a bijective function $f : \mathcal{V} \to \{0, 1, \ldots, V-1\}$ where $V = |\mathcal{V}|$ is the vocabulary size. The* ***encoder*** *applies f to a sequence of characters to yield a sequence of integers. The* ***decoder*** *applies f^{-1} to map the integers back to text.*

In these notes, we employ a *character-level* tokenizer. The set $\mathcal{V}$ consists of all unique characters present in the training dataset.

2.1 Mathematical Formulation of the Dataset

Let the raw dataset be a single string of length L. Through the tokenization mapping f, we obtain the fully tokenized dataset $\mathbf{D} \in \{0, 1, \ldots, V-1\}^L$.

To objectively measure the generalization of our model, $\mathbf{D}$ is strictly partitioned into two disjoint contiguous subsets:

1. **Training Set $\mathbf{D}_{\text{train}}$**: Typically the first 90% of $\mathbf{D}$. Used to update model parameters.
2. **Validation Set $\mathbf{D}_{\text{val}}$**: The remaining 10% of $\mathbf{D}$. Used strictly to compute the loss and measure overfitting.

3 Block Size and the Time Dimension

Neural networks do not process the entire dataset $\mathbf{D}_{\text{train}}$ at once. Instead, they process small, contiguous chunks of data.

Definition 3.1 (Block Size)**.** *The **block size** (also known as context length), denoted by T, is the maximum number of contiguous previous tokens the model is permitted to inspect to predict the subsequent token.*

Given a contiguous slice from the dataset of length $T + 1$, say $S = (s_0, s_1, \ldots, s_T)$, we can extract T distinct training examples. For $t \in \{1, 2, \ldots, T\}$, the input **context** is $(s_0, \ldots, s_{t-1})$ and the **target** is s_t.

Intuition: We train the model on sequences of length 1, length 2, all the way up to length T simultaneously. This ensures the neural network becomes accustomed to seeing contexts of varying lengths (from 1 up to T) during inference.

4 Batching in PyTorch

To maximize the utilization of parallel processing hardware (GPUs), we process multiple blocks simultaneously.

Definition 4.1 (Batch Dimension)**.** *The **batch size**, denoted by B, is the number of independent sequences processed concurrently in a single forward pass.*

Consequently, the input to the neural network is a tensor $\mathbf{X} \in \mathbb{N}^{B \times T}$ and the target tensor is $\mathbf{Y} \in \mathbb{N}^{B \times T}$. The elements of $\mathbf{Y}$ are simply the elements of $\mathbf{X}$ shifted by one time step into the future. If $\mathbf{X}_{b,t}$ is the token at index i in the original dataset, then $\mathbf{Y}_{b,t}$ is the token at index $i + 1$.

5 Worked Examples and Solved Exercises

5.1 Coding Example: Character-level Tokenizer

Problem: Given a string `text = "AABCAA"`, construct a character-level vocabulary, and implement the encoder and decoder functions in Python.

Solution:

```
        text = "AABCAA"

        # 1. Construct the vocabulary (unique characters, sorted)
        chars = sorted(list(set(text)))
        vocab_size = len(chars)
        print(f"Vocabulary: {chars}, Size: {vocab_size}")
        # Output: Vocabulary: ['A', 'B', 'C'], Size: 3

        # 2. Create the bijective mappings
        stoi = { ch:i for i,ch in enumerate(chars) }
        itos = { i:ch for i,ch in enumerate(chars) }

        # 3. Define encoder and decoder
        encode = lambda s: [stoi[c] for c in s]
        decode = lambda l: ''.join([itos[i] for i in l])

        # Test
        encoded_text = encode(text)
        print(encoded_text) # Output:[0, 0, 1, 2, 0, 0]
        print(decode(encoded_text)) # Output: "AABCAA"

```

5.2 Mathematical Exercise: Conditional Probabilities

Problem: Assume a vocabulary $\mathcal{V} = \{A, B\}$. We are given an autoregressive language model defined by the following conditional probabilities:

$$\begin{aligned} P(A) &= 0.6 \\ P(B \mid A) &= 0.8 \\ P(A \mid A, B) &= 0.5 \end{aligned}$$

Calculate the joint probability $P(X)$ of the sequence $X = (A, B, A)$.

Solution: Using the chain rule of probability for an autoregressive language model, the joint probability is factored as:

$$P(x_1, x_2, x_3) = P(x_1) \cdot P(x_2 \mid x_1) \cdot P(x_3 \mid x_1, x_2) \tag{2}$$

Substituting our sequence $X = (A, B, A)$:

$$P(A, B, A) = P(A) \cdot P(B \mid A) \cdot P(A \mid A, B) \tag{3}$$

We plug in the provided values:

$$P(A, B, A) = (0.6) \cdot (0.8) \cdot (0.5) = 0.24 \tag{4}$$

The probability of generating the sequence (A, B, A) is 0.24.

5.3 Coding Exercise: Extracting Batches

Problem: Let $\mathbf{D}_{\text{train}}$ be represented as a 1-dimensional PyTorch tensor of length L. Write a Python function `get_batch(data, B, T)` that randomly selects B starting indices and constructs the input tensor $\mathbf{X} \in \mathbb{N}^{B \times T}$ and the target tensor $\mathbf{Y} \in \mathbb{N}^{B \times T}$. Use PyTorch's `torch.stack` method.

Solution:

```
        import torch
        torch.manual_seed(1337) # For reproducibility

        def get_batch(data, B, T):
        """
        data: 1D torch.Tensor containing the tokenized dataset.
        B: Batch size (int)
        T: Block size / Context length (int)
        """
        # 1. Generate B random starting indices.
        # The maximum valid starting index is len(data) - T - 1 to ensure
        # we have enough tokens for both X (length T) and Y (length T shifted by
    1).
        ix = torch.randint(len(data) - T, (B,))

        # 2. Extract sequences for X. We slice from index i to i+T
        x = torch.stack([data[i : i+T] for i in ix])

        # 3. Extract sequences for Y. We slice from i+1 to i+T+1
        y = torch.stack([data[i+1 : i+T+1] for i in ix])

        return x, y

        # Verification
        dummy_data = torch.arange(20) # Tensor: [0, 1, ..., 19]
        X, Y = get_batch(dummy_data, B=4, T=8)

        print("Shape of X:", X.shape) # Output: torch.Size([4, 8])
        print("Shape of Y:", Y.shape) # Output: torch.Size([4, 8])

        # Observe the target Y is exactly X shifted by 1 step into the future
        print("First row of X:", X[0].tolist())
        print("First row of Y:", Y[0].tolist())

```